Ride-Sharing Application: An Object-Oriented Design Approach

Nisarg Pandya (202303103510361)

AMTICS, Uka Tarsadia University

Real-World Scenario

Tapping “Book Ride” feels like one action, but behind it a whole cast of objects cooperates. A rider requests a trip, the system picks a nearby driver, matches a vehicle – bike, auto or cab – estimates the fare, tracks the journey and finally settles the payment.

Every one of these actors behaves slightly differently, yet the app must treat them in one uniform way. This poster models that with **inheritance** and **polymorphism**, so a new vehicle type or payment mode never means rewriting the booking logic.

Problem Statement

A ride-sharing app runs into recurring design problems:

- Riders and drivers share many fields (name, phone, login) – copy-pasting them invites bugs
- Fare rules differ per vehicle type, so long **if-else** chains keep growing
- Payments come in many forms – cash, UPI, card – each with its own steps
- Every new vehicle or payment mode forces edits in code that already works

An object-oriented design gives each concern its own class, so the app can grow without turning into a tangle of conditionals.

OOD Focus: Inheritance & Polymorphism

This poster designs the object model of the app around two core ideas – each tied to an actual decision made for this project:

- **Inheritance** – `User` → `Rider / Driver`; `Vehicle` → `Bike / Auto / Cab`; `Payment` → `Cash / Upi / Card` (shared state is written once, in the parent)
- **Polymorphism** – each `Vehicle` subclass overrides `calculateFare()`; each `Payment` subclass overrides `pay()`; the right version runs at runtime
- **Abstraction** – `User`, `Vehicle` and `Payment` are abstract: they define a contract, not a real object
- **Association** – `Ride` links a Rider, a Driver, a Vehicle and a Payment (collaboration, not inheritance)

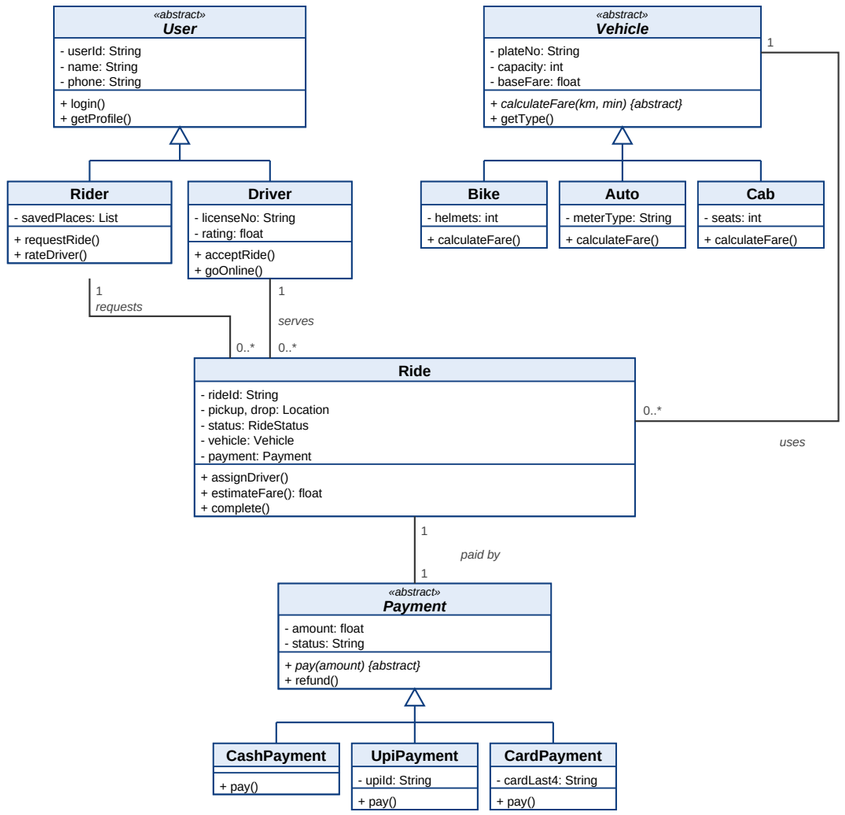
Objectives

- Model ride booking as a set of cooperating classes, not one monolithic script
- Use inheritance to move shared fields and behaviour into abstract parent classes
- Let `Ride` call `calculateFare()` and `pay()` without knowing the concrete type
- Replace type-checking if-else blocks with method overriding
- Show how the design stays open to new vehicles and payment modes

Principles Applied

- **Encapsulation** – rates, rating and ride status stay private behind public methods
- **Abstraction** – abstract parents hide the details of each concrete type
- **Liskov Substitution** – a Bike, Auto or Cab can stand in anywhere a Vehicle is expected
- **Open/Closed** – extend by adding subclasses, not by editing `Ride`
- **High cohesion, low coupling** – `Ride` depends only on abstract types

UML Class Diagram



Reading the model: `User`, `Rider` and `Driver` form one inheritance ladder; `Vehicle` with Bike, Auto and Cab forms a second, and `Payment` with its three modes a third. `Ride` is the hub – it holds a rider, a driver, a vehicle and a payment, but only through the **abstract parent types**. When `Ride` calls `vehicle.calculateFare()`, the version that runs depends on the real object behind it – that late binding is runtime polymorphism.

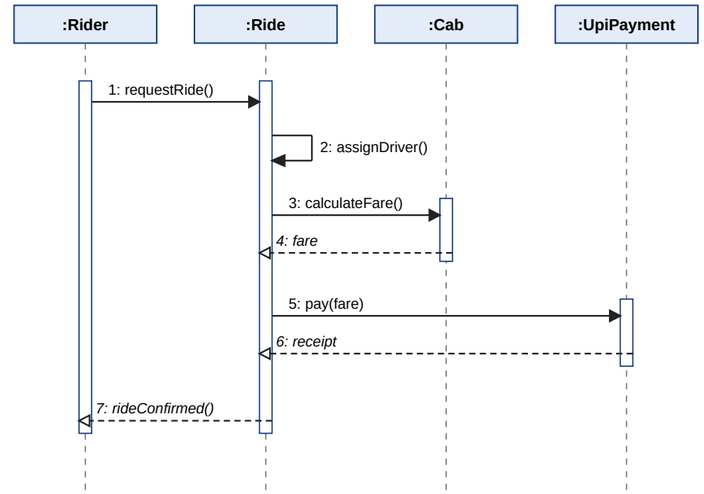
Future Scope

- **AI-based matching** – a demand-prediction model pairs riders with the best nearby driver and ETA
- **Dynamic pricing** – surge and discount rules added as new fare subclasses, with no change to `Ride`
- **Cloud deployment** – ride, payment and notification modules scaled independently as microservices
- **IoT & GPS** – vehicle sensors stream live location and speed into the `Vehicle` objects
- **EV & new vehicles** – an `ElectricScooter` is just one more `Vehicle` subclass
- **New payments** – `WalletPayment` or other modes plug in beside the existing three
- **Ride pooling & safety** – shared rides, SOS button and route-deviation alerts

Impact & Quality Assurance

- **Isolated testing** – each fare class is tested alone; `Ride` is tested with a stub vehicle returning a fixed fare
- **Maintainability** – a change in cab pricing touches the `Cab` class only
- **Open/Closed Principle** – a new `ElectricScooter` needs one extra subclass; `Ride` and `Payment` stay untouched
- **Reusability** – login and profile code is written once in `User` and reused by Rider and Driver
- **Scalability** – new cities or vehicle types add classes, not conditionals

Sequence Diagram: Polymorphism



Same call, different behaviour: the runtime type (:Cab, :Bike, :Auto) decides which calculateFare() runs – likewise for :UpiPayment / :CashPayment.

`Ride` only talks to the abstract `Vehicle` and `Payment` types. Swap one concrete object for another and the sequence stays identical – only the overridden method bodies change.

Conclusion

Building the ride-sharing app around abstract parents and specialised children – inheritance for shared state and behaviour, polymorphism for type-specific fares and payments – keeps the code testable today and easy to extend tomorrow, whether that means a new vehicle, a new payment mode or a new city.